

第3章: React入門

React Native の土台は **React** (リアクト) です。Reactの「コンポーネント (部品)」「props (プロップス)」「state (ステート)」という3つの考え方を理解すれば、スマホアプリの画面づくりがぐっと楽になります。この章では完全初心者向けに、これらを順番に解説します。

この章のコードについて: Reactの考え方を説明するため、一部Web向けの `<div>` などが出てきますが、**React Native**では `<View>` などに置き換わるだけで考え方は完全に同じです (対応は第4章で表にします)。まずは"考え方"に集中してください。

1. React とは — 画面を「部品」で組み立てる

React は、画面を コンポーネント (component = 部品) という小さなパーツに分けて作るための技術 (ライブラリ) です。Meta社 (旧Facebook) が開発しました。

身近な例え (レゴブロック) : 小さなブロック (コンポーネント) を組み合わせて、家や車 (画面全体) を作るイメージです。一度作ったブロックは別の場所でも再利用できます。「書籍カード」というブロックを1つ作れば、一覧画面で何個でも使い回せます。

「ライブラリ (library)」とは? 「特定の機能を提供するプログラムの部品集」のこと。図書館 (library) から本を借りるように、便利な機能を借りて使えます。Reactは「画面づくりの機能」を提供するライブラリです。

2. コンポーネント — 画面の部品

2.1 一番シンプルなコンポーネント

コンポーネントは「何かの見た目を返す関数」です。第2章で学んだ関数の知識がそのまま使えます。

```
// この一行は「React Nativeから Text という部品を借りてくる」という意味（詳細は第4章）
import { Text } from "react-native";

// Greeting : コンポーネント名。コンポーネント名は必ず大文字始まりにする決まり
// () => { ... } : 第2章で学んだアロー関数。引数なしの関数
// この関数は「画面に表示する内容」を return で返す
function Greeting() {
  return <Text>こんにちは!</Text>; // return の後ろが「見た目」。<Text>...</Text> が画面に
  出る文字
}

export default Greeting; // export default : この部品を「外のファイルから使えるように公開す
る」宣言
```

コンポーネント名はなぜ大文字始まり？ Reactは「小文字始まり＝HTMLの普通のタグ」「大文字始まり＝自作コンポーネント」と区別します。だから自作の部品は必ず **Greeting** のように大文字で始めます。これはルールなので守りましょう。

import と **export** とは？ - **import**（インポート）：他のファイルや部品集から「機能を借りてくる」命令。 - **export**（エクスポート）：自分のファイルの機能を「外に公開する」命令。ファイル同士で部品を貸し借りするための仕組みです。

2.2 JSX — **<Text>こんにちは</Text>** の正体

return の後ろに書いた **<Text>こんにちは!</Text>** のような、HTMLに似た記法を **JSX（ジェイエスエックス）** と呼びます。「JavaScriptの中に、画面の見た目を直接書ける特別な記法」です。

```
// JSXの基本ルール
return (
  <View>                                {/* <View> は「箱」。中に複数の部品をまとめられる（第4章で詳
  説） */}
    <Text>1冊目の本</Text>              {/* タグは <開始>中身</終了> の形 */}
    <Text>2冊目の本</Text>
  </View>
);
// 複数行のJSXは（ ）で囲む / 一番外側は1つのタグ（ここでは<View>）でまとめる必要がある
// { /* ... */ } はJSXの中でのコメントの書き方
```

JSX内の **{ }**（中カッコ） — 変数を埋め込む: JSXの中で **{ }** を使うと、その中にTypeScriptの値や式を埋め込みます。

```
const name = "鈴木";
return <Text>こんにちは、{name}さん</Text>;
// { name } : 中カッコの中はTypeScriptの世界。変数 name の中身 ("鈴木") が文字列に埋め込まれる
// 画面表示: こんにちは、鈴木さん
```

3. props — 部品に「材料」を渡す

同じ「書籍カード」でも、表示する本のタイトルは1冊ごとに違います。このようにコンポーネントの外から渡す材料を props (プロップス、propertiesの略) と呼びます。

```
import { View, Text } from "react-native";

// BookCard : 書籍カードのコンポーネント
// props (材料) として title と author を受け取る形を、型で定義する
type BookCardProps = {
  title: string;    // 本のタイトル (文字列)
  author: string;   // 著者名 (文字列)
};

// ({ title, author }) : 受け取ったpropsから title と author を取り出す書き方 (分割代入という)
// : BookCardProps : 「受け取るpropsはこの型ですよ」と宣言
function BookCard({ title, author }: BookCardProps) {
  return (
    <View>
      <Text>{title}</Text>    {/* 受け取った title を表示 */}
      <Text>{author}</Text>   {/* 受け取った author を表示 */}
    </View>
  );
}

export default BookCard;
```

このBookCardを使う側は、HTMLの属性のようにpropsを渡します。

```
// title= と author= に渡したい値を指定する。これがpropsとしてBookCardに届く
<BookCard title="リーダブルコード" author="Dustin Boswell" />
<BookCard title="達人プログラマー" author="David Thomas" />
// 同じBookCard部品を、違う材料(props)で2回使い回している！ これがコンポーネントの便利さ
```

「分割代入（ぶんかつだいにゆう）」とは？ `{ title, author }` のように、オブジェクトの中から必要な項目だけを取り出して変数にする書き方です。 `props.title` と毎回書く代わりに、最初に `title` だけ取り出しておけば短く書けます。Reactでは非常によく使います。

`props`は「読み取り専用」: 子コンポーネントは受け取った`props`を書き換えてはいけません。あくまで「親から渡された材料を表示するだけ」と考えてください。値を変化させたいときは、次に学ぶ `state` を使います。

4. state — 部品が持つ「変化する値」

4.1 state とは

`state`（ステート＝状態）は、コンポーネントが内部に持つ「変化する値」です。たとえば「検索ボックスに入力された文字」「ボタンが押された回数」「読込中かどうか」などです。`props`が「外から渡される材料」なのに対し、`state`は「自分の中で変わっていく値」です。

4.2 `useState` の使い方

`state`を作るには `useState`（ユーズ・ステート）という機能（フック）を使います。

```
import { useState } from "react"; // Reactから useState を借りてくる
import { View, Text, Button } from "react-native";

function Counter() {
  // useState(0) : 初期値0のstateを作る
  // 戻り値は配列で [今の値, 値を変える関数] の2つ。分割代入で受け取る
  // count      : 今の値 (最初は0)
  // setCount    : countを変更するための専用関数 (名前はset+state名が慣習)
  const [count, setCount] = useState(0);

  return (
    <View>
      <Text>押した回数: {count}</Text>           {/* 今のcountを表示 */}
      <Button
        title="押す"                             // ボタンに表示する文字
        onPress={() => setCount(count + 1)}       // onPress : 押されたときの処理 / setCount
        でcountを+1する
      />
    </View>
  );
}
```

useState の戻り値が配列なのはなぜ？ **useState** は「今の値」と「値を変える関数」の2つをセットで返します。 `const [count, setCount] = useState(0)` は、その2つを配列として受け取り、**count** と **setCount** という名前を付けているのです。名前は自由ですが、変更関数は **set + 変数名** にするのが慣習です。

4.3 stateを直接書き換えてはいけない

```
const [count, setCount] = useState(0);

count = count + 1;           // ✗ これはダメ！ 直接書き換えても画面は更新されない
setCount(count + 1);        // ○ 必ず専用の関数（setCount）を使う。これで画面も自動更新される
```

なぜ専用関数を使う必要があるの？ Reactは「stateが変わったら画面を描き直す」という仕組みで動いています。**setCount** を呼ぶことで初めてReactが「変化した！画面を更新しよう」と気づきます。直接 `count = ...` と書くと、Reactは変化に気づけず、画面が古いままになります。これは初心者がつまづきやすい超重要ポイントです。

5. フック（Hooks） — Reactの便利機能

useState のように **use** で始まる機能を **フック（Hooks）** と呼びます。コンポーネントに様々な能力を追加する道具です。代表的なものを紹介します（後の章で使います）。

フック	役割	主な登場章
useState	変化する値（state）を持つ	全章
useEffect	画面表示時などのタイミングで処理を実行する	第7章
useRouter / useLocalSearchParams	画面遷移やパラメータの取得（Expo Router）	第6・8章

5.1 **useEffect** のさわり

useEffect（ユーズ・エフェクト）は「特定のタイミングで処理を実行する」フックです。代表的な使い方が「画面が最初に表示されたとき、サーバーからデータを取ってくる」処理です。

```
import { useEffect, useState } from "react";

function BookList() {
  const [books, setBooks] = useState([]); // 本のリストをstateで持つ（最初は空の配列[]）

  // useEffect(実行したい処理, 依存配列) の形
  useEffect(() => {
    // この中に「画面表示時に1回だけやりたい処理」を書く（例：データ取得）
    console.log("画面が表示されました");
  }, []);
  // 第2引数の [] : 「依存配列」。空の[]は「最初の1回だけ実行する」という意味になる

  return <Text>本の数: {books.length}</Text>; // books.length : 配列の要素数（本の冊数）
}
```

「依存配列（いそんはいれつ）」とは？ `useEffect` の2つ目の引数 `[]` のことです。「この中の値が変わったら処理を再実行する」という指定です。空の `[]` にすると「最初の1回だけ」になります。第7章で実際にSupabaseからデータを取得する際に詳しく使います。今は「画面表示時の処理は `useEffect` に書く」とだけ覚えればOKです。

6. コンポーネントを組み合わせる

Reactの本質は「小さなコンポーネントを組み合わせて大きな画面を作る」ことです。先ほどのBookCardを使って、一覧画面を組み立ててみましょう。

```
import { View, Text } from "react-native";

// 子コンポーネント：1冊分のカード（再掲）
type BookCardProps = { title: string; author: string };
function BookCard({ title, author }: BookCardProps) {
  return (
    <View>
      <Text>{title}</Text>
      <Text>{author}</Text>
    </View>
  );
}

// 親コンポーネント：一覧画面。BookCardを並べて使う
function BookListScreen() {
  return (
    <View>
      <Text>書籍一覧</Text>
      {/* 同じBookCardを、異なるpropsで何度も並べる */}
      <BookCard title="リーダブルコード" author="Dustin Boswell" />
      <BookCard title="達人プログラマー" author="David Thomas" />
      <BookCard title="TypeScript入門" author="鈴木 僚太" />
    </View>
  );
}

export default BookListScreen;
```

親子関係: `BookListScreen`（親）が `BookCard`（子）を使っています。親が子にpropsで材料を渡す、という関係です。実際のアプリでは、本のデータは配列で管理し、第2章で学んだ `map` を使って自動的にカードを並べます（第7章で実装します）。

7. Web の React と React Native の違い（予告）

この章のコードは「考え方」を説明するためのもので、実際のReact Nativeでは部品の名前が少し変わります。次章への橋渡しとして、対応表を載せておきます。

役割	WebのReact (HTML)	React Native
箱・まとまり	<code><div></code>	<code><View></code>
文字の表示	<code><p></code> , <code></code>	<code><Text></code>
ボタン	<code><button></code>	<code><Button></code> , <code><Pressable></code>
入力欄	<code><input></code>	<code><TextInput></code>
画像	<code></code>	<code><Image></code>

重要: コンポーネント・props・state・フックという「考え方」は、WebでもReact Nativeでもまったく同じです。違うのは「使う部品の名前」と「文字は必ず `<Text>` で囲む」などのモバイル特有のルールだけです。次章でその違いを実際に手を動かして学びます。

8. この章のまとめ

- **コンポーネント:** 見た目を返す関数。名前は大文字始まり。これを組み合わせて画面を作る
- **JSX:** `<Text>...</Text>` のように見た目を書く記法。`{ }` で変数を埋め込める
- **props:** 部品の外から渡す「材料」。読み取り専用
- **state:** 部品が内部に持つ「変化する値」。`useState` で作り、変更は必ず `setXxx` 関数で
- **フック:** `use` で始まる便利機能。`useState` (状態)、`useEffect` (タイミング処理) など
- これらの考え方は **WebでもReact Nativeでも共通**。違うのは部品の名前

次の章へ: いよいよReact Native本体です。第4章では、`View` / `Text` / `FlatList` などスマホ専用の部品と、画面を配置するFlexbox、そしてExpo Routerによる画面遷移を学びます。